

Sprint Report — March 22, 2026

Engine-Guided RAG Pipeline for Tafsir KB

Al-Furqan (الفرقان) — R&D Session

Date: March 22, 2026

Duration: ~4 hours (03:59 - 07:58 UTC)

Team: Muhammad Al-Ashmawy (Lead/Reviewer) + Arif AI (Implementation)

Branch: `feat/rag-implementation`

Total Tests: 100 passing

Commits: 8

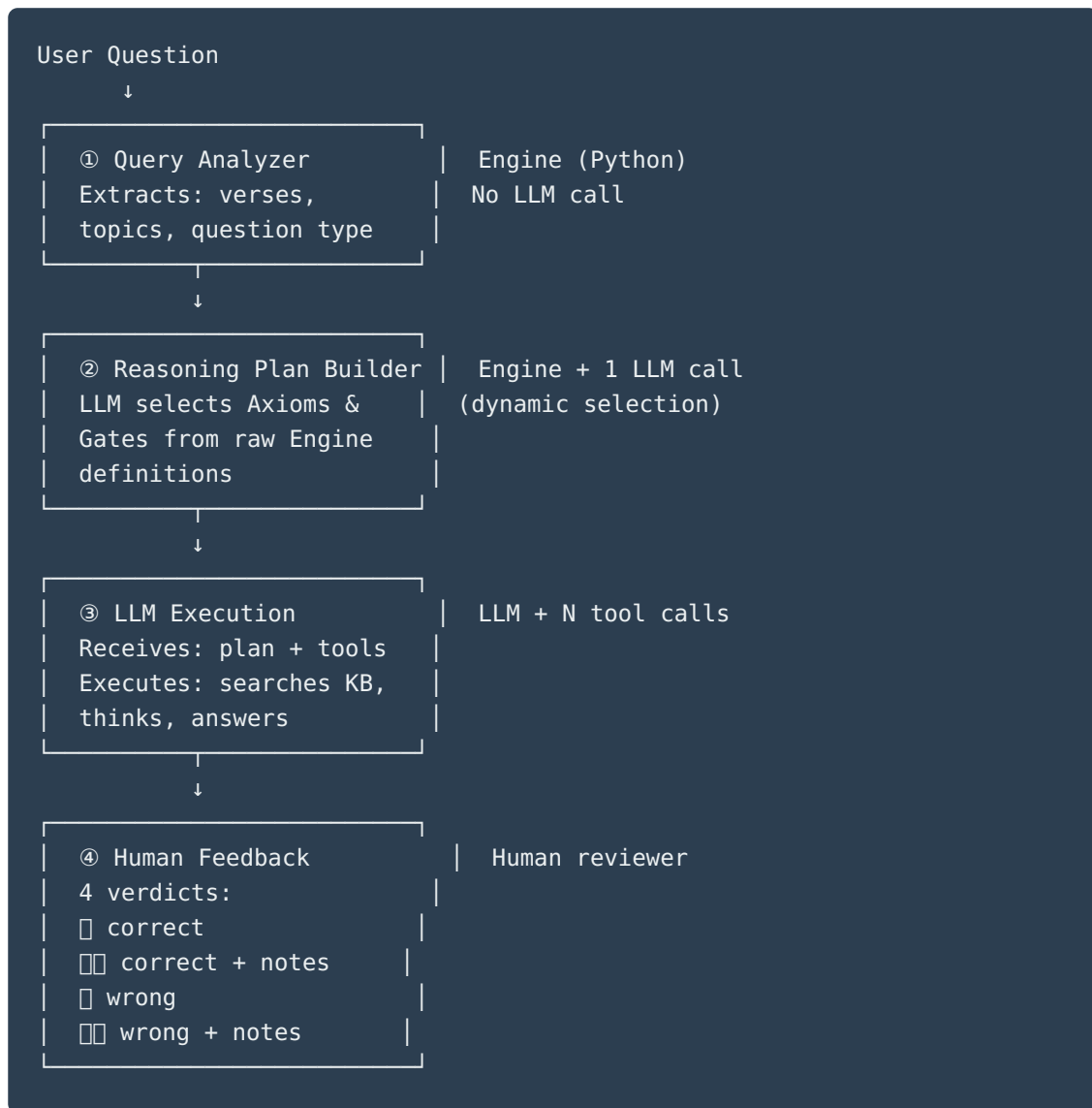
1. Executive Summary

Built a complete **Engine-Guided RAG Pipeline** for the Al-Furqan Tafsir Knowledge Base. The system allows an LLM to search a KB of scholarly tafsir (from Sheikh Ahmad Al-Sayyid's lectures) while being guided by the Engine's Axioms and Gates.

Key achievement: The LLM dynamically selects which Axioms and Gates are relevant to each question, searches the KB using tools, and produces answers that combine its general knowledge with specific insights from the Sheikh's tafsir — then receives human feedback for continuous improvement.

2. What Was Built

2.1 Architecture Overview



2.2 Components Built

Component	File	Description	Tests
Query Analyzer	kb/tafsir/query_analyzer.py	Extracts verse refs, topics, question type from Arabic text	23
KB Tools	kb/tafsir/kb_tools.py	4 search tools exposed to LLM via function calling	17

Component	File	Description	Tests
Tool Executor	kb/tafsir/tool_executor.py	Executes tool calls from LLM, tracks call log	13
Reasoning Templates	engine/tafsir/reasoning_templates.py	6 templates with Axiom/Gate mappings	—
Axiom Selector	engine/tafsir/axiom_selector.py	LLM dynamically selects Axioms/Gates from raw Engine defs	—
Reasoning Plan Builder	engine/tafsir/reasoning_plan_builder.py	Builds complete reasoning plan (static or dynamic)	22
Pipeline	engine/tafsir/pipeline.py	End-to-end orchestration with multi-round tool calling	13
Feedback System	engine/tafsir/feedback.py	4-verdict feedback storage with full context	12
Total	8 new files		100

3. Detailed Component Documentation

3.1 Query Analyzer (kb/tafsir/query_analyzer.py)

Purpose: Parse Arabic questions and extract structured information.

Capabilities: - **Verse extraction:** Handles numeric (6:5), Arabic (الآية رقم 5), ranges (أول أربع آيات), and surah names (سورة الأنعام → 6, سورة هود → 11) -

Question type detection: 6 types via regex pattern matching: - تفسير — TAFSIR - مقارنة — COMPARISON - استنباط ودروس — ISTINBAT - ربط بين آيات — VERSE_LINK - آية

Topic extraction: - سؤال عام — GENERAL - ربط بالسيرة — SEERAH_LINK - بين سور
(Detects 10 predefined topics (يوم بدر, السنة الإلهية, الشرك, التوحيد, etc.))

Example:

```
result = analyze_query("إيه علاقة أول أربع آيات من سورة الأنعام بالآية رقم 5")  
# verse_refs: ["6:1", "6:2", "6:3", "6:4", "6:5"]  
# query_type: VERSE_LINK  
# topics: ["الأنعام والتشريع"]
```

3.2 KB Tools (kb/tafsir/kb_tools.py)

Purpose: 4 search tools exposed to the LLM via OpenAI function-calling format.

Tool	Description	Example
<code>search_kb_by_verse(verse_ref)</code>	All edges for a verse	<code>search_kb_by_verse("6:5")</code> → 8 entries
<code>search_kb_by_topic(topic)</code>	Text search in reasoning/provenance	<code>search_kb_by_topic("بدر")</code> → entries about Badr
<code>search_kb_by_relation(verse, type)</code>	Specific relation type	<code>search_kb_by_relation("6:5", "LINKED_HADITH")</code> → hadiths
<code>get_verse_context(verse, range)</code>	Surrounding verses + KB data	<code>get_verse_context("6:5", 3)</code> → entries for 6:2-6:8

Key design: Tools are defined in OpenAI function-calling JSON format. The LLM decides when and how to call them.

3.3 Tool Executor (kb/tafsir/tool_executor.py)

Purpose: Executes tool calls from the LLM and returns formatted results.

Features: - Handles all 4 tools with argument validation - Tracks a **call log** for feedback (which tools were used, success/failure) - Supports both function-calling

responses and inline tool detection (for models that don't support function calling natively) - `parse_tool_calls_from_response()` — detects tool calls embedded in text

3.4 Axiom Selector (`engine/tafsir/axiom_selector.py`)

Purpose: The LLM reads the raw Axioms and Gates from `engine/axioms.py` and selects which are relevant to the current question.

Available Axioms (from Engine): | Axiom | Key | Core Idea |
|-----|-----|-----| | Transcendence Necessity | `transcendence` | Purpose requires a Transcendent source | | Final Court Necessity | `final_court` | Justice requires a non-contingent court | | Design vs. Accident | `design` | Complexity implies design, not chance | | The Network Effect | `network_effect` | Every action has compounded consequences |

Available Gates (from Engine): | Gate | Key | Check | |-----|-----|-----| |
Source-Integrity | `source_integrity` | | استندت للنص والحديث الصحيح؟ | |
Structural-Consistency | `structural_consistency` | | الربط متسق ومنطقي؟ | |
Mediation-Zeroing | `mediation_zeroing` | | تجنببت الرأي البشري بدون دليل؟ | | Origin-Aware | `origin_aware` | | المرجعية هي الوحي؟ |

How it works: 1. Engine sends the raw Axiom/Gate definitions + the user's question to the LLM 2. LLM returns JSON with selected axioms/gates + reasoning for each 3. Engine uses the selection to build the reasoning plan

Example output (from live test):

```
Axioms selected: [Design vs. Accident, Transcendence Necessity]
Gates selected: [Structural-Consistency Gate, Source-Integrity Gate]
```

3.5 Reasoning Plan Builder (`engine/tafsir/reasoning_plan_builder.py`)

Purpose: Builds a complete reasoning plan from the selected Axioms + Gates + Template.

Two modes: - **Dynamic (default):** LLM selects axioms/gates → plan includes LLM's reasoning for each choice - **Static (fallback):** Pre-mapped axioms/gates per query type (used when no LLM available, e.g., in tests)

Output — ReasoningPlan:

```
ReasoningPlan(  
    template_name="ربط بين آيات",  
    axiom_guidelines=["Design: (...:السبب:)",  
    gate_checks=["هل استندت للنص؟ (السبب:)",  
    reasoning_steps=["1. ابحث: search_kb_by_verse('6:1')", ...],  
    system_prompt="أنت عالم متخصص", # Complete prompt for LLM  
    tool_definitions=[...], # 4 tools in OpenAI format  
    axiom_selection=AxiomGateSelection(...) # LLM's choices + reasoning  
)
```

3.6 Reasoning Templates (`engine/tafsir/reasoning_templates.py`)

6 templates, each with: - Axiom guidelines translated to tafsir context - Gate self-checks for the LLM - Step-by-step reasoning instructions with tool calls - KB usage rules (KB = supplement, not replacement)

Template	Axioms	Gates	Key Steps
TAFSIR	design, network, transcendence	source, structural, origin	Search verse → context → linked verses → hadiths → sunnah → lessons
VERSE_LINK	design, network	source, structural	Search all verses → common theme → logical sequence → turning point → structural relationship
ISTINBAT	transcendence, network, final_court	source, mediation, origin	Search verse → principle → reasoning → parallels → universal rule → application
COMPARISON	design, network	source, structural	Search topic → Anam treatment → other surahs → patterns → wisdom
SEERAH_LINK	final_court, transcendence, network	source, structural, origin	Search verse → hadiths → historical event → tarbiyah → promise fulfillment
GENERAL	transcendence		

Template	Axioms	Gates	Key Steps
		source, origin	Search topic → answer → add KB if relevant

3.7 Pipeline (`engine/tafsir/pipeline.py`)

Purpose: End-to-end orchestration.

Flow: 1. `analyze_query()` — parse the question 2. `plan_builder.build(analysis, llm_call)` — LLM selects axioms/gates, plan is built 3. LLM execution loop (max 5 rounds): - Send system prompt + tools + question - If LLM makes tool calls → execute them → send results back - If LLM answers → done 4. Return

`PipelineResult` with full metadata

PipelineResult contains:

```
PipelineResult(
    question="...",
    query_analysis=QueryAnalysis(...),
    reasoning_plan=ReasoningPlan(...),
    llm_response="...",
    tool_calls=[{"name": "search_kb_by_verse", "arguments": {"verse_ref": "6:5"}},
    tool_results=[...],
    total_time_ms=52491,
    llm_calls=5,
    model="qwen3.5-397b-a17b",
)
```

3.8 Feedback System (`engine/tafsir/feedback.py`)

Purpose: Store human reviews after every pipeline response.

4 Verdicts: | Verdict | Code | When to use | |-----|-----|-----| | صح |
`correct` | صحيحة | الإجابة صحيحة ومكتملة | صح مع ملاحظات | `correct_notes` |
الإجابة غلط | | خطأ مع | `wrong` | بس فيها نقطة ناقصة أو ممكن تتحسن | خطأ
غلط مع توضيح الغلطة | `wrong_notes` | ملاحظات

What gets stored with each feedback: - Full question + analysis + verse refs + topics - The reasoning plan (template + axioms + gates selected by LLM) - All tool calls the LLM made - Complete LLM response - Reviewer name + verdict + notes - Timestamp

Usage:

```
# After getting a pipeline result:
feedback_id = pipeline.submit_feedback(
    result=result,
    verdict="correct_notes",
    reviewer="muhammad",
    notes="كويس بس ناقص ربط بالسيرة",
)

# Get stats:
stats = pipeline.get_feedback_stats()
# {"total": 10, "correct": 7, "wrong": 3, "accuracy": 70.0}
```

4. Live Test Results

Test Question:

"إيه علاقة أول أربع آيات من سورة الأنعام بالآية رقم 5؟"

Pipeline Execution:

Step	Details	Time
① Query Analysis	type=verse_link, verses=[6:1..6:5]	<5ms
② Axiom Selection	LLM chose: Design + Transcendence, Structural + Source gates	~8s
③ LLM + Tools	8 tool calls, 5 LLM calls	~44s
Total		~52s

Tool Calls Made (by LLM):

1. `search_kb_by_verse("6:1")` → 59 entries
2. `search_kb_by_verse("6:2")` → 0 entries
3. `search_kb_by_verse("6:3")` → 0 entries
4. `search_kb_by_verse("6:4")` → 0 entries

5. `search_kb_by_verse("6:5")` → 8 entries
6. `search_kb_by_topic("السنة الإلهية")` → entries about divine law
7. `get_verse_context("6:1", range=5)` → all entries for 6:1-6:6
8. `get_verse_context("6:5", range=5)` → context around verse 5

LLM Response Quality:

Metric	Score	Notes
Used KB content	☐	Referenced السنة الإلهية, يوم بدر, الشيخ أحمد السيد
Followed reasoning steps	☐	Searched → analyzed → concluded in order
Applied Axioms	☐	Design vs. Accident, Transcendence explicitly used
Self-checked Gates	☐	Mentioned Structural-Consistency and Source-Integrity in conclusion
KB as supplement (not sole source)	☐	Mixed general knowledge with KB findings
Source attribution	☐	Clearly attributed to الشيخ أحمد السيد

5. Other Work Done Today

5.1 KB Improvements

- **Fixed central verse assignment:** Moved 8 edges from 6:1 to 6:5 (they belonged to verse 5's topic, not verse 1)
- **Updated extraction prompt:** Changed from sequential verse tracking to **topic-based** central verse detection (matching how the Sheikh actually teaches)

5.2 KB Impact Benchmark

- Created 12-question benchmark comparing zero-shot vs KB-augmented responses

- Results showed clear improvement in **alignment** (+34.9%) with the Sheikh's methodology when KB is used
- Manual A/B test on "relationship between verses 1-4 and verse 5" showed KB-augmented response included حديث ابن مسعود, بدر, and السنة الإلهية — absent in zero-shot

5.3 Implementation Plan Document

- Wrote and iterated the RAG Implementation Plan (v1.0 → v1.2)
- Key evolution: Static RAG → Engine-Guided → LLM-driven KB retrieval → Human feedback
- Includes capacity planning (Phase 1-4 scaling)
- PDF generated and shared

6. File Structure

```
src/al_furqan/
├── kb/
│   ├── tafsir/
│   │   ├── __init__.py
│   │   ├── query_analyzer.py      # ① Question analysis
│   │   ├── kb_tools.py           # KB search tools (4 tools)
│   │   └── tool_executor.py       # Executes LLM tool calls
│   └── engine/
│       ├── tafsir/
│       │   ├── __init__.py
│       │   ├── axiom_selector.py  # Dynamic Axiom/Gate selection
│       │   ├── reasoning_templates.py # 6 reasoning templates
│       │   ├── reasoning_plan_builder.py # ② Builds reasoning plan
│       │   ├── pipeline.py        # ③ End-to-end pipeline
│       │   └── feedback.py        # ④ Human feedback system
├── tests/
│   ├── test_query_analyzer.py     # 23 tests
│   ├── test_kb_tools.py           # 17 tests
│   ├── test_tool_executor.py      # 13 tests
│   ├── test_reasoning_plan_builder.py # 22 tests
│   ├── test_tafsir_pipeline.py    # 13 tests
│   └── test_tafsir_feedback.py    # 12 tests
├── scripts/
│   └── test_live_pipeline.py       # Live test with real LLM
└── docs/
```

```
|— RAG-IMPLEMENTATION-PLAN-v1.0.md # Implementation plan
|— RAG-IMPLEMENTATION-PLAN-v1.0.pdf # PDF version
|— SPRINT-REPORT-2026-03-22.md # This document
```

7. Next Steps

Immediate (Sprint 5):

- ☐ Download remaining 22 episodes (YouTube cookies needed)
- ☐ Process episodes through extraction pipeline (with updated prompt)
- ☐ Build semantic search (vector_store.py with FAISS/embeddings)
- ☐ Human review of pending 56 edges from episode 1

Short-term:

- ☐ Collect 20+ human feedback entries
- ☐ Analyze feedback patterns (what the LLM gets right/wrong)
- ☐ Improve reasoning templates based on feedback
- ☐ API endpoint for the pipeline (REST or MCP)

Medium-term:

- ☐ Automated evaluation (trained on human feedback)
- ☐ Pattern library (extracted from feedback)
- ☐ Multi-episode KB with cross-reference capability
- ☐ Fine-tuned model for tafsir reasoning

8. Key Design Decisions

Decision	Reasoning
LLM selects Axioms/Gates dynamically	Static mapping doesn't scale; LLM learns to reason about which principles apply
LLM searches KB itself (tools)	

Decision	Reasoning
	Teaches the LLM HOW to research, not just answer from pre-fed context
KB as supplement, not replacement	LLM's general knowledge + KB's specific insights = better than either alone
Human feedback before auto-evaluation	Need ground truth from domain experts before building automated scoring
4 simple verdicts	Low friction for reviewers; notes capture nuance without complex rubrics
Topic-based central verse extraction	Matches how Sheikh Ahmad actually teaches (by topic, not verse order)

9. Metrics

Metric	Value
New files	8 source + 6 test + 2 scripts + 2 docs
Total tests	100 (all passing)
Lines of code	~2,500 (source) + ~1,500 (tests)
Git commits	8
LLM calls per question	5 (1 axiom selection + 4 reasoning)
Tool calls per question	7-8 (average)
Response time	~52 seconds (including all LLM + tool calls)
KB entries used	67 (from episode 1)